FORMOSA CRYPTO

Completing the Chain:

Verified Implementations of Hash-Based Signatures and Their Security

Manuel Barbosa François Dupressoir **Rui Fernandes**
Andreas Hülsing Matthias Meijers Pierre-Yves Strub

Repository

Paper

CHES 2026

TCHES 2026(3), pp. 958–981

Jasmin [ABB+17, ABB+20]

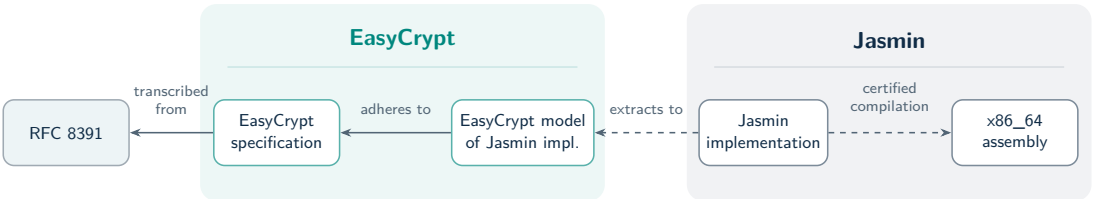
Jasmin [ABB+17, ABB+20]

- Stack zeroization [OBG+24]
- (Static) Safety checker based on abstract interpretation
- Compiler correctness proved in Rocq
- Extraction to EasyCrypt
- Speculative constant-time:
  - Spectre v1 [SBG+23] and other spectre variants [ABC+25]
  - Compilation preserves SCT [ABB+25].

Compiler Correctness & Constant-Time Preservation

Compilation preserves the semantics of *safe* source programs and constant-time behavior: functional correctness and constant-time guarantees established at source level carry to the assembly level.

Verification Workflow



Trusted Computing Base

- Correctness of the SHA-256 implementation is outside the scope of the proof

```
op Hash : W8.t list -> nbytes. (* Abstract operator *)

axiom hash_128 (x : W8.t Array128.t):
  phoare[
    M(Syscall).hash_plen_3n____sha256:
    arg.`2 = x
    ==>
    to_list res = NBytes.val (Hash (to_list x))
  ] = 1%r.
```

Figure: EasyCrypt axiom stating the correctness of SHA-256.

Trusted Computing Base

- Jasmin compiler: unverified compilation passes (e.g. parsing, pretyping, etc.)
- Jasmin extraction to EasyCrypt
- SMT Solvers (Z3, CVC4 and Alt-Ergo)

Implementation: Performance

	C Ref	Jasmin (abs, %)	loop zeroization (abs, %)	unrolled zeroization (abs, %)
Keygen	10092.74	8263.54 (-18.12%)	8293.61 (+0.36%)	8312.35 (+0.59%)
Sign	20251.01	24775.17 (+22.34%)	24872.05 (+0.39%)	24927.53 (+0.61%)
Verify	10.35	8.47 (-18.14%)	8.48 (+0.13%)	8.95 (+5.69%)

Table: Performance Comparison between C and Jasmin implementations in 10^6 clock cycles (column #2). Zeroization overhead w.r.t. Jasmin baseline (columns #3 and #4).

QUESTIONS?

References I

[ABB+17] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Arthur Blot, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Hugo Pacheco, Benedikt Schmidt, and Pierre-Yves Strub.
Jasmin: High-assurance and high-speed cryptography.
In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1807–1823. ACM Press, October / November 2017.

[ABB+20] José Bacelar Almeida, Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, Vincent Laporte, Tiago Oliveira, and Pierre-Yves Strub.
The last mile: High-assurance and high-speed cryptographic implementations.
In *2020 IEEE Symposium on Security and Privacy*, pages 965–982. IEEE Computer Society Press, May 2020.

[ABB+25] Santiago Arranz-Olmos, Gilles Barthe, Lionel Blatter, Benjamin Grégoire, and Vincent Laporte.
Preservation of speculative constant-time by compilation.
Proc. ACM Program. Lang., 9(POPL):1293–1325, 2025.

References II

[ABC+25] Santiago Arranz-Olmos, Gilles Barthe, Chitchanok Chuengsatiansup, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Peter Schwabe, Yuval Yarom, and Zhiyuan Zhang.

Protecting cryptographic code against spectre-rsb: (and, in fact, all known spectre variants).

In Lieven Eeckhout, Georgios Smaragdakis, Katai Liang, Adrian Sampson, Martha A. Kim, and Christopher J. Rossbach, editors, *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, pages 933–948. ACM, 2025.

[OBG+24] Santiago Arranz Olmos, Gilles Barthe, Ruben Gonzalez, Benjamin Grégoire, Vincent Laporte, Jean-Christophe Léchenet, Tiago Oliveira, and Peter Schwabe.

High-assurance zeroization.

IACR TCHES, 2024(1):375–397, 2024.

References III

[SBG+23] Basavesh Ammanaghatta Shivakumar, Gilles Barthe, Benjamin Grégoire, Vincent Laporte, Tiago Oliveira, Swarn Priya, Peter Schwabe, and Lucas Tabary-Maujean.

Typing high-speed cryptography against spectre v1.

In *2023 IEEE Symposium on Security and Privacy*, pages 1094–1111. IEEE Computer Society Press, May 2023.